

Nitesh's Personal Java OOP Revision Guide (2+ Years Product Company Prep)

This is not a generic OOP document. It is based on the exact confusions, questions, and mental models developed during your Java revision. Goal: - Quick revision before interviews - Remember WHY Java behaves a certain way - Avoid the same confusion points - Focus on product-company style thinking

1. Encapsulation

Data Hiding != Encapsulation
Data Hiding: - Make variables private. - Prevent direct access.
Encapsulation: - Put data + rules together. - Control access through methods and constructors.
Your memory line: 'Private variable is data hiding. Private + rules + validation = encapsulation.'
BankAccount example: withdraw() validates amount before changing balance.

2. Constructors (Your Biggest Learning)

Question you asked: 'Why not use methods instead of constructors?' Answer: Constructors guarantee an object starts in a valid state. Your memory line: 'Constructor makes sure a variable is not an orphan. If an Employee must have a name, force it in the constructor.' Example: Employee emp = new Employee("Nitesh"); Good use: Required fields -> constructor Optional fields -> setter

3. Immutability

Your confusion: 'If value is final, how can constructor assign it?' Rule: final means assign only once.
Valid: private final int balance; Account(int balance){ this.balance = balance; } Invalid: Assign again in another method. Memory line: 'final = one assignment only, not necessarily at declaration.'

4. Inheritance

Meaning: IS-A relationship Manager IS-A Employee Dog IS-A Animal Inheritance is for sharing common behavior and data. Do not use inheritance only to reuse code. Use it only when relationship genuinely exists.

5. Private Variables in Inheritance

Your confusion: Why child cannot access private variables? Rule: private belongs only to the class where it is declared. Child class cannot directly access it. Solution: Getter Setter Protected (carefully)

6. Runtime Polymorphism (Most Important)

Your biggest confusion point. Example: Employee e = new Developer(); Memory line: Reference Type Controls Visibility. Object Type Controls Execution. Visible methods: Employee methods only Executed method: Developer's overridden implementation Interview gold line: 'Compile time checks reference type. Runtime execution depends on object type.'

7. Why Child Methods Are Not Accessible?

Employee e = new Developer(); e.code(); // Error Why? Because compiler only sees Employee reference. Even though object is Developer, reference controls visible methods.

8. Method Overloading vs Overriding

Overloading: Same method name Different parameters Compile-time polymorphism Overriding: Same method signature Child changes implementation Runtime polymorphism Quick memory: Overloading = many forms before runtime Overriding = many forms at runtime

9. Abstract Class vs Interface (Major Weak Area)

Your breakthrough understanding: Abstract Class: WHAT YOU ARE Employee Animal Vehicle
BankAccount Interface: WHAT YOU CAN DO Flyable Swimmable PaymentService
NotificationService Use Abstract Class when: - Shared state exists - Shared implementation exists
Use Interface when: - Only behavior/contract matters

10. Multiple Inheritance Question You Raised

Your observation was correct. Diamond Problem alone is NOT the full reason. Real issue: - State conflicts - Constructor conflicts - Initialization order conflicts - Method conflicts Interfaces are safer because they mostly define behavior.

11. Interface Revision

Interface can have: - Abstract methods - Default methods - Static methods - Private methods (Java 9+) Interface cannot have: - Instance variables - Constructors Memory line: 'Interface defines capability, not ownership of state.'

12. Product Company Interview Traps

Q: Why String is immutable? Q: Why multiple inheritance not allowed? Q: Why interfaces in Spring?
Q: Why constructor injection? Q: Difference between IS-A and HAS-A? Q: Reference vs Object?
Prepare these extremely well.

13. 5-Minute Last Minute Revision

Encapsulation: Private + validation + rules Constructor: Guarantees valid object creation
Inheritance: IS-A relationship Polymorphism: Reference controls visibility Object controls execution
Abstract Class: What you ARE Interface: What you CAN DO final: Assign once private: Accessible
only inside same class Most Important Interview Topic: Employee e = new Developer(); Understand
this completely.